Give the team a ready workplace

What changed after your feedback, and what the new research session can establish.

The earlier team was asked to preserve baseline melody without receiving the prediction files. Later engineers repeatedly restored software dependencies instead of testing recognition ideas. We have changed those handoffs: the lead now declares the required artifacts, the host supplies verified files, and the experiment engineer starts in a tested, reusable environment.

We followed the approved implementation sequence

First, make the assignment executable. Required baseline predictions are inventoried, checked for missing or empty files, copied into the assignment, and verified by hash. The engineer gets a read-only prediction directory. The reference answers stay with the evaluator.

Second, prepare and reuse the workplace. A versioned recipe builds Python notation and OCR tools. A fresh restricted container must export synthetic MusicXML to a nonempty kern file, read four notes back, and recognize synthetic chord text before the host marks it ready. Later engineers reuse its immutable image with separate writable workspaces.

Third, give environment problems an owner. Routine setup requires no model call. An environment engineer is consulted for a missing dependency or a failed readiness test. It proposes bounded package changes; the host builds and tests them. An experiment engineer can report a missing capability and stop instead of spending the research budget installing software.

Fourth, retry the research from scratch. New sessions use new state, assignments and workspaces. The same development images and declared baseline artifacts are legitimate inputs. Prepared Docker layers may be reused, but that must be disclosed when reporting setup time.

> **The original completion rule remains:** a seven-excerpt pilot cannot finish the 32-excerpt goal. Full completion still requires the configured melody/chord thresholds, parsing and transposition checks, audited chord timing and musician review. A successful environment test is not a successful transcription.

Clear ownership, enforced handoffs

Each person has one primary job. Communication goes through recorded assignments and results.

Team member	Responsibility and handoff
Research lead	Choose a hypothesis, declare the inputs, assign a measurable task, and decide the next branch from observed results. It can request environment preparation but does not install packages.
Environment engineer	Diagnose package requirements with a separate, small context. Return a validated package proposal and a reason. It cannot run host commands, change the base image or readiness test, access labels, or declare accuracy.
Experiment engineer	Write and run recognition code in the prepared container. Inspect inputs, test outputs and submit source plus predictions. Escalate missing capabilities through a structured request.
Host and review desk	Deterministic code validates contracts, controls Docker, saves evidence and budgets, freezes submissions, and scores predictions against protected labels. These are not additional language-model agents.
Human reviewer	Review the implementation PR, evidence and limitations; request changes before merging. No automated comment-to-code or auto-merge loop was added.

Principles borrowed from the Claude Code analysis

The implementation applies typed tool contracts, separate role contexts, initialization before task execution, bounded output summaries, and durable handoff records. Full build logs stay on disk; the model sees the first actionable diagnostic and a short tail. No third-party Claude Code source was copied.

The tradeoff is deliberate

This initial environment supports Python notation and OCR, plus bounded package additions. It does not promise Audiveris, GPUs, arbitrary Dockerfiles or downloaded model bundles. A new capability needs a reviewed readiness test. This limits flexibility, but avoids claiming that a generic import check proves a new recognition method can execute.

The base Python dependencies are hash-locked and image reuse is immutable. New transitive or Debian dependencies are recorded after resolution; every future rebuild is not guaranteed byte-for-byte identical. Environment retention also consumes disk space.

What verification already caught

Fresh research is necessary, alongside focused checks of the execution contracts.

The deterministic checks

The regression suite passed 87 tests, with three Docker checks run separately. CI passed both the regression and Docker jobs. Real containers demonstrated protected scoring/isolation, a complete synthetic export plus OCR, separate writable workspaces reusing one prepared image, and read-only baseline/system files. These checks used no research-model calls.

The first live attempt exposed an image lifecycle bug

The lead supplied 14 baseline artifacts to the first experiment. Setup completed in about 2.3 seconds with Docker layers already warm. After two slow model calls, the engineer tried its first command and Docker reported that the handed-off image was missing. It requested environment help and did not invent predictions.

The fault was ours: separate registries could rebuild the same recipe and move its shared image tag. On this Docker installation, the previous image became unavailable. We stopped and preserved that attempt, gave every publication a unique retained tag, included user/group identity in the cache key, and added a real two-registry regression test. Both CI jobs passed after the fix.

The comparison also encountered model latency

A fresh restart with the original gpt-6-astra model reached dataset inspection, baseline review and source reading, but repeatedly timed out while deciding the first experiment. We preserved and interrupted that attempt. A separate gpt-6.1-sol decision probe succeeded; its proposed action was not executed. We then began a distinct fresh research session with that model.

How to interpret the comparison: changing the model prevents attributing a research outcome to the environment changes alone. The new session tests the workflow end to end. Both interrupted attempts and diagnostic calls belong in the cost record, even though neither produced an accuracy measurement. Dollar billing is unavailable.

The research retry did not reach scoring

The engineering change is implemented; the research outcome is still unverified.

What the first engineer actually received and did

The lead asked for conservative chord recognition from enlarged image crops while preserving the supplied enlarged-Audiveris melody. The assignment required seven parseable outputs, a chord provenance table, explicit timing uncertainty and a transposition round-trip check. Fourteen baseline artifacts were delivered. This repaired the missing-input handoff from the original session.

The engineer tried to inspect the prepared workspace, received the missing-image failure, saved a preflight script, and requested restoration of the environment. It produced no transcription files. The lead did not receive a score it could use to improve recognition. We interrupted the session to fix image publication, rather than allowing further experiments to inherit that host failure.

What happened after the fix

The original-model restart completed its initial reviews but recorded three decision timeouts, including two consecutive timeouts choosing the first experiment. The alternate-model session completed the same initial reviews, then recorded two consecutive experiment-decision timeouts. We stopped both and preserved their checkpoints and interruption records. No experiment engineer was dispatched in either post-fix session.

Session	Calls reserved / attempted	Wall time to stop	New scored result
Initial v2 attempt	11	12.4 min	None
Post-fix, original model	8	13.7 min	None
Post-fix, alternate model	7	8.9 min	None

Two additional diagnostic calls took about 73 seconds combined. Across research attempts and probes, 28 calls were reserved or attempted. This is not 28 completed responses. Interrupted pending work is missing from checkpoint active time, so the table uses observed wall time to operator stop. Monetary billing and complete timeout token usage are unavailable.

Did accuracy improve?

There is no new measurement. The original session had reduced chord errors from 30/38 to 25/38, but lost the melody and produced 0/7 parseable complete files. This follow-up produced no evaluated candidate, so it cannot establish an accuracy gain, a regression, or an evidence-driven change of recognition approach.

Review status: implementation PR #3 remains a draft. Automated checks pass, but the planned live comparison is inconclusive. The user should not be asked to approve a claim that the research system now transcribes better.

What is established, and what remains

A reviewable engineering result with an explicit research-verification gap.

Question	Evidence and conclusion
Are labels missing?	No. The supplied reference files remain unchanged and available to the evaluator. Recognition engineers receive predictions and images, not evaluation answers.
Can engineers reuse a ready environment?	Yes in real-container verification: two separate workspaces reused the prepared image. The image-publication regression also passed across two independent registries.
Did setup become cheaper?	The first profile validation recorded 42.3 setup seconds; the first live build recorded 2.3 seconds with layers warm. This is not a controlled cold-build speed comparison. The old run did not separate setup time reliably.
Did the environment engineer repair a real research dependency?	Not demonstrated in these runs. The first engineer escalated correctly; the host image bug was repaired by us. Bounded package-repair behavior is covered by tests.
Did fresh research improve transcription?	Unknown. No new predictions were scored. The post-fix sessions stalled on model decisions before research execution.
Did the protection checks remain intact?	Post-fix checkpoint audits passed: original files, protected artifacts, evidence records, budgets and completion consistency. No goal was marked complete.

The next verification should retain this discipline

First make model-decision failures diagnosable: the current provider saves the prompt and timeout exception, but does not retain partial stream output on timeout. Establish reliable completion on the actual experiment-decision prompt, then run another fresh session without injecting a hand-written experiment or recycling a better result from earlier attempts.

Use the same model and total budget for a clean comparison where practical. Report cold setup and cached reuse separately. Require real predictions, independent scoring and a subsequent evidence-based decision before calling the research loop verified. New songs are needed for any held-out generalization claim.

Where the review evidence lives

Implementation: PR #3, stacked on the design PR #2. Protected implementation revision: c578de9. The original attempt belongs to b30dcdd; its old contract was preserved. The three run folders contain prompts, completed model receipts, decisions and interruption notes. A machine-readable verification summary records counts and limitations. The post-fix audit files describe checkpoint integrity, not successful research completion.

The durable checkpoints still say running because they were interrupted before the runner wrote a final state; each interruption note records the actual stopped status. No research process is left running. The initial attempt must be inspected against its own implementation revision, not silently rescored with the new protected code.

References: github.com/WhaSukGO/researchsquad/pull/3; docs/environment-operations.md; docs/design/environment-and-review.md; runs/environment-followup-verification-20261028.json. The design document contains the Claude Code source analysis and primary Anthropic references.